



UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN
HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 9-2

NOMBRE COMPLETO: Uriarte Ortiz Enrique Yahir

N° de Cuenta: 318234757

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: Sabado 26 de Octubre del 2024.

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas.

- 1) Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHC MONOPOLY. animado desplazándose las letras de izquierda a derecha como si fuera letrero LCD/LED de forma cíclica.

Para esta actividad utilice la tipografía que encontré en internet del universo de mi personaje para el proyecto final, en GIMP recorte cada letra y la acomode en una nueva imagen para formar la palabra solicitada y la exporte como **TNProy.png** a la carpeta de Texturas del proyecto de la práctica.



Fig. 1. Tipografía del universo de Phineas y Febr.

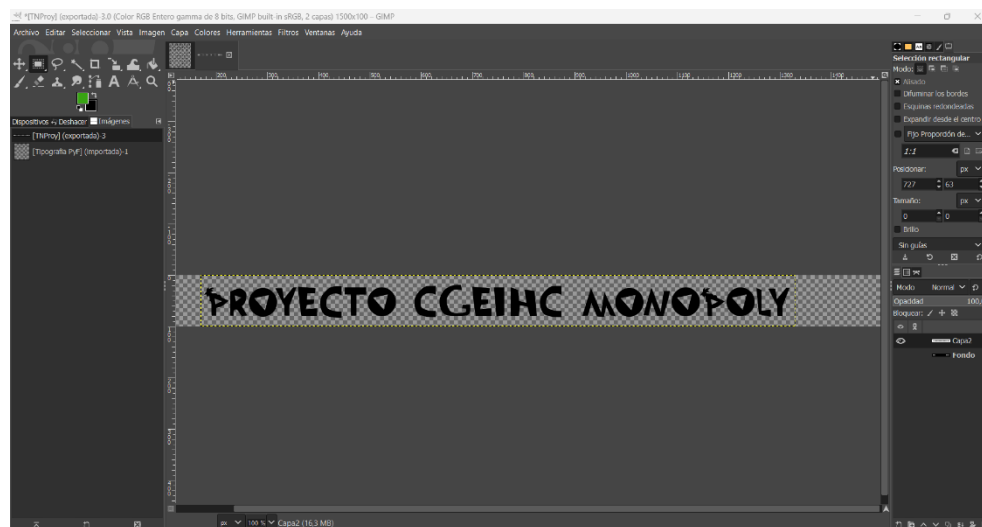


Fig. 2. Palabra con la tipografía de mi universo.

En el código main definí las variables, los modelos y las texturas para esta actividad, tuve que modificar el movimiento para mi letrero ya que como lo había realizado en la actividad original reiniciaba el tiempo y con eso en esta actividad se trababa más el movimiento del letrero. La función de cada variable es la siguiente: **movLetrero** movimiento del letrero arriba abajo, **movOffset** para la velocidad de movimiento del letrero, **rotLetrero** para la rotación del letrero, **toffsetnumerov** y

toffsetnumerocambiau para el movimiento de la imagen la frase, **ultimaActivacion** para verificar el tiempo transcurrido de 2 segundos para cada movimiento y **avanza2** es una bandera para verificar el movimiento del letrero.

También definí una nueva figura, un rectángulo ajustado en dimensiones para la palabra que tendrá el movimiento e indiqué la ubicación de la textura y los modelos.

```
//Variables de Actividad 01.
float movLetrero, movOffset, rotLetrero;
float toffsetnumerov, toffsetnumerocambiau;
float ultimaActivacion;
bool avanza2;

Model Arco;
Model Letrero;

Texture PalabraTexture;
```

Fig. 3. Variables para la actividad.

```
unsigned int numeroIndices[] = { 0,1,2, 0,2,3 };
GLfloat numeroVertices[] = {
    -5.45f, 0.0f, 2.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    5.45f, 0.0f, 2.5f, 0.25f, 0.0f, 0.0f, -1.0f, 0.0f,
    5.45f, 0.0f, -2.5f, 0.25f, 1.0f, 0.0f, -1.0f, 0.0f,
    -5.45f, 0.0f, -2.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f };

Mesh* obj7 = new Mesh();
obj7->CreateMesh(numeroVertices, numeroIndices, 32, 6);
meshList.push_back(obj7);
```

Fig. 4. Rectángulo para la palabra.

```
PalabraTexture = Texture("Textures/TNProy.png");
PalabraTexture.LoadTextureA();

Arco = Model();
Arco.LoadModel("Models/Arco.obj");
Letrero = Model();
Letrero.LoadModel("Models/Letrero.obj");
```

Fig. 5. Ubicación de modelos y textura.

```
//Variables de Actividad 01.
movLetrero = 0.0; movOffset = 0.4f; rotLetrero = 0.0f;
toffsetnumerov = 0.0f; toffsetnumerocambiau = 0.0f;
ultimaActivacion = 0.0f;
avanza2 = true;
```

Fig. 6. Inicialización de Variables.

```
rotLetrero += 3.5f * deltaTime;

// Movimiento del letrero.
if ((now - ultimaActivacion) >= 2.0f && avanza2 == true) {
    if (movLetrero > -41.0f) {movLetrero -= movOffset * deltaTime; rotLetrero = 0;}
    else {avanza2 = false; ultimaActivacion = now; rotLetrero = 0;} }
else if ((now - ultimaActivacion) >= 2.0f && avanza2 == false) {
    if (movLetrero < 0.0f) {movLetrero += movOffset * deltaTime; rotLetrero = 0;}
    else { avanza2 = true; ultimaActivacion = now; rotLetrero = 0; } }
```

Fig. 7. Movimiento del Letrero.

Para las modificaciones del movimiento del letrero indique la velocidad de rotación del letrero, después en su mayoría son los mismo ifs de la actividad original solo que aquí en lugar de verificar que hayan pasado 2 segundos desde la ultima vez que se reinició **glfwGetTime()**;, verifica que hayan pasado 2 segundos desde su último activación y el valor de la bandera **avanza2**, si es **true** esta arriba, si es **false** esta abajo, realiza el movimiento correspondiente, les da nuevos valores a las variables empleadas e indica que el letrero de una vuelta con la variable correspondiente. Después defino los modelos del arco y letrero, como en la actividad original, y defino el rectángulo con el texturizado, ya que el movimiento es de izquierda a derecha, cambio el signo positivo a negativo y le asigno una velocidad más baja para que se pueda ver, después le asigno en la translación y rotación los mismos movimientos que el modelo del letrero, asegurándome primero que este en la misma ubicación del modelo, de modo que tenga el efecto que la palabra si está en el letrero.

```
//Arco.
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Arco.RenderModel();

//Letrero
model = glm::translate(model, glm::vec3(0.0f, movLetrero + 48.6f, 5.0f));
if (rotLetrero <= 360.0f) { model = glm::rotate(model, rotLetrero * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); }
else { model = glm::rotate(model, 360.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); }
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Letrero.RenderModel();

glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

toffsetnumerocambiau -= 0.0025 * deltaTime;
if (toffsetnumerocambiau > 1.0) toffsetnumerocambiau = 0.0;
toffsetnumerov = 0.0;
toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, movLetrero/3.4 + 12.5f, 1.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
if (rotLetrero <= 360.0f) { model = glm::rotate(model, rotLetrero * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); }
else { model = glm::rotate(model, 360.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); }
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
PalabraTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[6]->RenderMesh();
```

Fig. 8. Movimiento de palabra del Letrero.

- 2) Separar las cabezas del Dragón y agregar las siguientes animaciones:
 - a) Movimiento del cuerpo ida y vuelta.
 - b) Aleteo
 - c) Cada cabeza se mueve de forma diferente de acuerdo con una función/algorithmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc.)
 - d) Cada cabeza debe de verse de un color diferente: roja, azul, verde, blanco, café.

Para comenzar esta segunda actividad separe en 3ds Max cada cabeza del modelo, las alas ya estaban separadas gracias a la actividad de clase, además para que se viera mejor el dragón lo separe de la base.

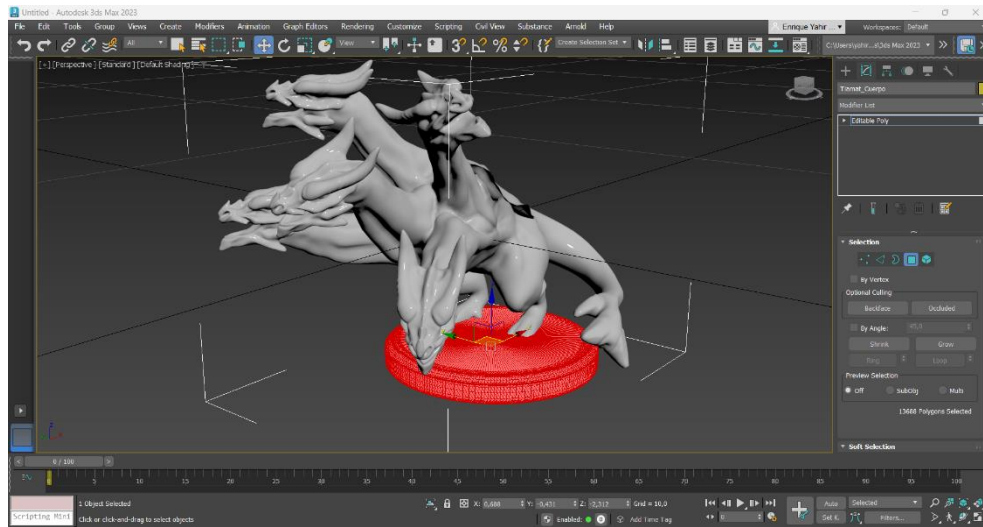


Fig. 9. Separando Base del Cuerpo.

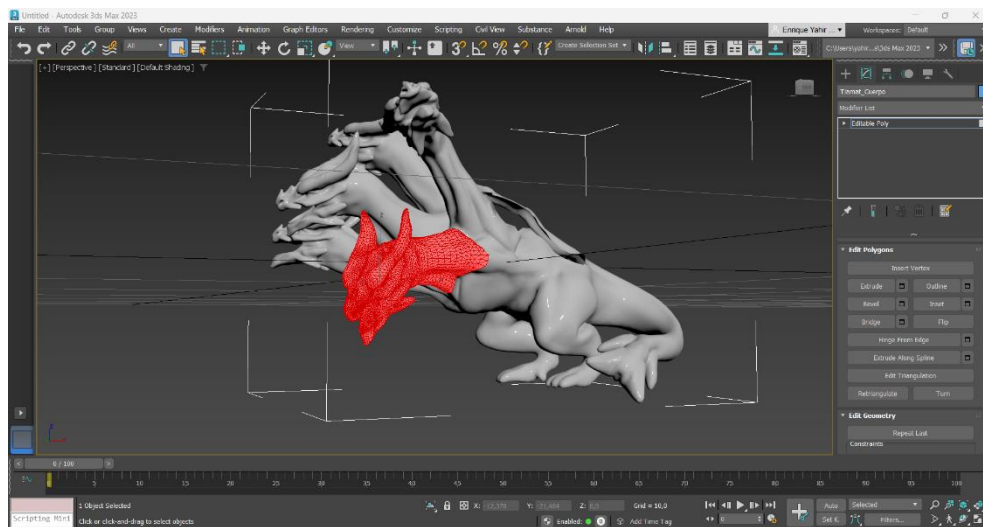


Fig. 10. Separando Cabeza 2 del Cuerpo.

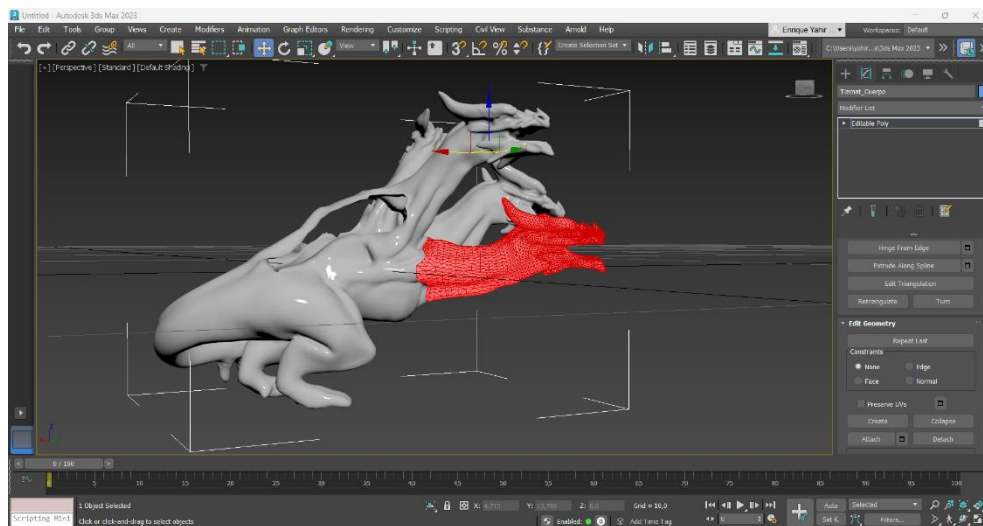


Fig. 11. Separando Cabeza 3 del Cuerpo.

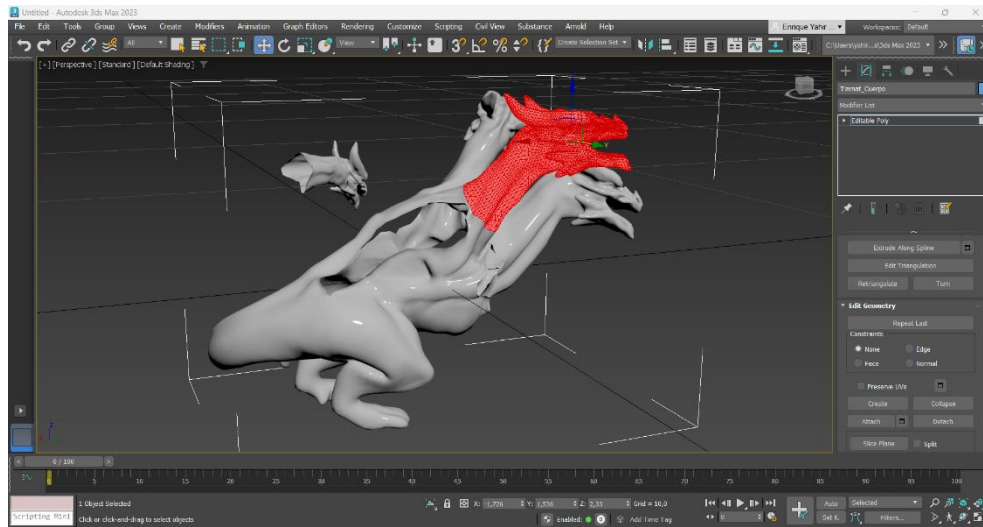


Fig. 12. Separando Cabeza 4 del Cuerpo.

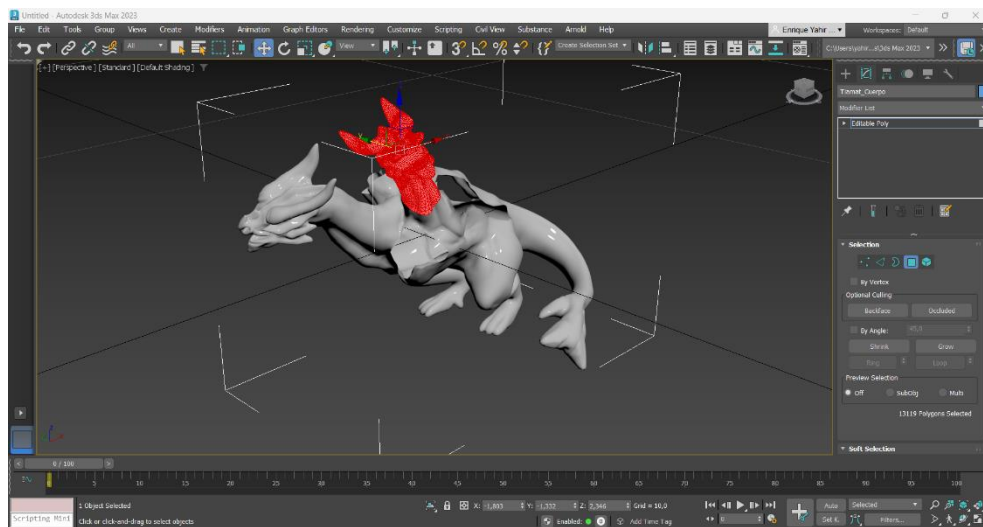


Fig. 13. Separando Cabeza 5 del Cuerpo.

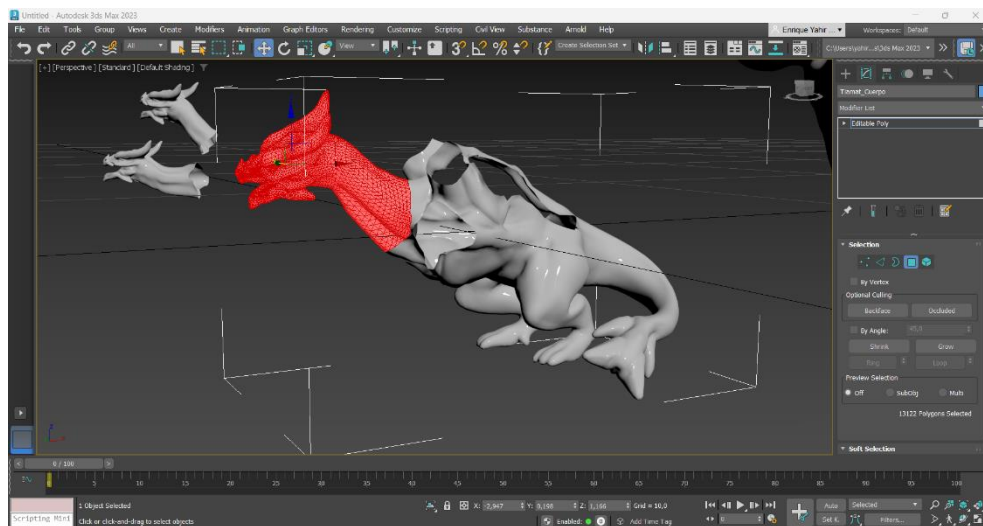


Fig. 14. Separando Cabeza 1 del Cuerpo.

En el código main declare las variables para esta actividad y los modelos para esta actividad. La función de cada variable es la siguiente: **movD** para el movimiento del dragón por completo, **movDOffset** para velocidad de movimiento, **giro** para asignarle el giro de 180° y valla de regreso el modelo, **avanza** como una bandera para indicar el inicio del movimiento, **ida** para indicar el sentido en el que se mueve, si su valor es **true** va hacia la izquierda y si el valor cambia a **false** va hacia la derecha, las demás variables son para signar el movimiento en los ejes X y Y de acuerdo a la figura de cada función, las funciones que escogió fueron, la cabeza 1 se mueve como una *Espiral de Arquímedes*, la cabeza 2 se mueve como una *Roseta*, la cabeza 3 se mueve como una *Lemniscata de Bernoulli*, la cabeza 4 se mueve como una *Cardioide*, y la cabeza 5 se mueve como una *Hipocicloide*.

```
//Variables de Actividad 02.
float movD, movDOffset, giro;
bool avanza, ida;

float r, espiralX, espiralY; // Espiral de Arquimedes.
float rosaX, rosaY; // Roseta.
float lemniscataX, lemniscataY; // Lemniscata de Bernoulli.
float cardioideX, cardioideY; // Cardioide.
float hipocicloideX, hipocicloideY; // Hipocicloide.

Model Tiamat_Cuerpo;
Model Tiamat_Ala01; Model Tiamat_Ala02;
Model Tiamat_Cabeza01; Model Tiamat_Cabeza02;
Model Tiamat_Cabeza03; Model Tiamat_Cabeza04;
Model Tiamat_Cabeza05;
```

Fig. 15. Variables para la actividad.

```
Tiamat_Cuerpo = Model();
Tiamat_Cuerpo.LoadModel("Models/Tiamat_Cuerpo.obj");
Tiamat_Ala01 = Model();
Tiamat_Ala01.LoadModel("Models/Tiamat_Ala01.obj");
Tiamat_Ala02 = Model();
Tiamat_Ala02.LoadModel("Models/Tiamat_Ala02.obj");
Tiamat_Cabeza01 = Model();
Tiamat_Cabeza01.LoadModel("Models/Tiamat_Cabeza01.obj");
Tiamat_Cabeza02 = Model();
Tiamat_Cabeza02.LoadModel("Models/Tiamat_Cabeza02.obj");
Tiamat_Cabeza03 = Model();
Tiamat_Cabeza03.LoadModel("Models/Tiamat_Cabeza03.obj");
Tiamat_Cabeza04 = Model();
Tiamat_Cabeza04.LoadModel("Models/Tiamat_Cabeza04.obj");
Tiamat_Cabeza05 = Model();
Tiamat_Cabeza05.LoadModel("Models/Tiamat_Cabeza05.obj");
```

Fig. 16. Ubicación de modelos y textura.

```
//Variables de Actividad 02.
movD = 0.0f; movDOffset = 0.3f; giro = 0.0f;
avanza = true; ida = true;

r = 0.0f; espiralX = 0.0f; espiralY = 0.0f; // Espiral de Arquimedes.
rosaX = rosaY = 0.0f; // Roseta.
lemniscataX = lemniscataY = 0.0f; // Lemniscata de Bernoulli.
cardioideX = cardioideY = 0.0f; // Cardioide.
hipocicloideX = hipocicloideY = 0.0f; // Hipocicloide.
```

Fig. 17. Inicialización de Variables.

Para el movimiento del dragon de ida y vuelta, el código funciona de la siguiente manera, si la variable avanza es true indica que ya se puede empezar a mover, después si ida es true avanza hacia la izquierda hasta -100 X, cuando alcanza esa

distancia, cambia de valor de ida a false e indica que a giro sea de 180° a modo que el modelo de la vuelta, cuando ida es false ejecuta el else, que es lo mismo solo que el movimiento es hacia la derecha.

```
// Movimiento del dragon (ida y vuelta).
if (avanza == true) {
    if (ida == true) {
        if (movD > -100.0f) {movD -= movDOffset * deltaTime;}
        else { ida = false; giro += 180.0f; } }
    else {
        if (movD < 100.0f) {movD += movDOffset * deltaTime;}
        else { ida = true; giro += 180.0f; } }
    angulovaria += 0.5f * deltaTime;
}
```

Fig. 18. Movimiento del dragon.

Después definí la ecuación para el movimiento de las cabezas, a continuación, mostrare la ecuación en la que me base para cada una, en la bibliografía están los enlaces donde saque las ecuaciones explicó cada una:

I. Espiral de Arquímedes.

Ecuaciones originales: $r = a + b\theta$ $x = r * \cos(\theta)$ $y = r * \sin(\theta)$

Ecuaciones en código: $r = 2.0 + 0.01 * \text{angulovaria};$

$\text{espiralX} = r * \cos(\text{angulovaria} * 0.5f);$

$\text{espiralY} = r * \sin(\text{angulovaria} * 0.5f);$

Siendo el radio inicial de la espiral de 2, la tasa de crecimiento de la espiral de 0.1 y el 0.5f para controlar su velocidad de crecimiento.

II. Curva de Rosa (Roseta).

Ecuaciones originales: $x = a * \cos(k\theta) * \cos(\theta)$ $y = a * \cos(k\theta) * \sin(\theta)$

Ecuaciones en código: $k = 3.0f; a = 30.0f;$

$\text{rosaX} = a * \cos(k * \text{angulovaria} * 0.05f) * \cos(\text{angulovaria} * 0.05f);$

$\text{rosaY} = a * \cos(k * \text{angulovaria} * 0.05f) * \sin(\text{angulovaria} * 0.05f);$

Siendo k el número de pétalos, a el tamaño y el 0.5f para controlar su velocidad de crecimiento.

III. Lemniscata de Bernoulli.

Ecuaciones originales: $x = (a * \cos(t)) / (1 + \sin^2(t))$ $y = (a * \sin(t) * \cos(t)) / (1 + \sin^2(t))$

Ecuaciones en código: $a_lemn = 30.0f;$

$\text{lemniscataX} = (a_lemn * \cos(\text{angulovaria} * 0.05f)) / (1 + \sin(\text{angulovaria} * 0.05f) * \sin(\text{angulovaria} * 0.05f));$

$\text{lemniscataY} = (a_lemn * \sin(\text{angulovaria} * 0.05f) * \cos(\text{angulovaria} * 0.05f)) / (1 + \sin(\text{angulovaria} * 0.05f) * \sin(\text{angulovaria} * 0.05f));$

Siendo a_lemn para definir el tamaño de la lemniscata y el 0.5f para controlar su velocidad de crecimiento.

IV. Cardioide.

Ecuaciones originales: $x = a(2\cos(t) - \cos(2t))$ $y = a(2\sin(t) - \sin(2t))$

Ecuaciones en código: $a_card = 15.0f; // Escala$

$\text{cardioideX} = a_card * (2 * \cos(\text{angulovaria} * 0.1f) - \cos(2 * \text{angulovaria} * 0.1f));$

$\text{cardioideY} = a_card * (2 * \sin(\text{angulovaria} * 0.1f) - \sin(2 * \text{angulovaria} * 0.1f));$

Siendo a_card para controlar el tamaño del cardioide y el 0.1f para controlar su velocidad de crecimiento.

V. Hipocicloide.

Ecuaciones originales: $x = (R-r)\cos(t) + r\cos((R-r)t/r)$ $y = (R-r)\sin(t) - r\sin((R-r)t/r)$

Ecuaciones en código: $R = 20.0f$; $r = 5.0f$;

$\text{hipocicloideX} = (R - r) * \cos(\text{angulovaria} * 0.1f) + r * \cos((R - r) * \text{angulovaria} * 0.1f / r)$;

$\text{hipocicloideY} = (R - r) * \sin(\text{angulovaria} * 0.1f) - r * \sin((R - r) * \text{angulovaria} * 0.1f / r)$;

Siendo R el radio del círculo exterior fijo, r el radio del círculo interior que rueda, (R - r) define la distancia desde el centro al punto de contacto, la relación R/r determina cuántas cúspides tendrá la curva y el 0.1f para controlar su velocidad de crecimiento.

```
// Funcion Espiral de Arquimedes.
r = 2.0 + 0.01 * angulovaria;
espiralX = r * cos(angulovaria * 0.5f);
espiralY = r * sin(angulovaria * 0.5f);

// Funcion Roseta (Curva de Rosa), 3 pétalos.
float k = 3.0f; // Numero de pétalos
float a = 30.0f; // Escala
rosaX = a * cos(k * angulovaria * 0.05f) * cos(angulovaria * 0.05f);
rosaY = a * cos(k * angulovaria * 0.05f) * sin(angulovaria * 0.05f);

// Funcion Lemniscata de Bernoulli.
float a_lemn = 30.0f; // Escala
lemniscataX = (a_lemn * cos(angulovaria * 0.05f)) / (1 + sin(angulovaria * 0.05f))
              * sin(angulovaria * 0.05f);
lemniscataY = (a_lemn * sin(angulovaria * 0.05f) * cos(angulovaria * 0.05f)) / (1 + sin(angulovaria * 0.05f))
              * sin(angulovaria * 0.05f);

// Funcion Cardioides.
float a_card = 15.0f; // Escala
cardioidex = a_card * (2 * cos(angulovaria * 0.1f) - cos(2 * angulovaria * 0.1f));
cardioidy = a_card * (2 * sin(angulovaria * 0.1f) - sin(2 * angulovaria * 0.1f));

// Funcion Hipocicloide.
float R = 20.0f; // radio del círculo fijo
float r = 5.0f; // radio del círculo móvil
hipocicloideX = (R - r) * cos(angulovaria * 0.1f) + r * cos((R - r) * angulovaria * 0.1f / r);
hipocicloideY = (R - r) * sin(angulovaria * 0.1f) - r * sin((R - r) * angulovaria * 0.1f / r);
```

Fig. 19. Funciones para cada cabeza del dragon.

Para tener mejor las posiciones de los elementos emplee jerarquías, **modelaux** para la jerarquía general del cuerpo y las alas, las demás jerarquías cada una es respectivamente para una cabeza.

```
glm::mat4 model(1.0);
glm::mat4 modelaux(1.0);
glm::mat4 JerC1(1.0); glm::mat4 JerC2(1.0); glm::mat4 JerC3(1.0);
glm::mat4 JerC4(1.0); glm::mat4 JerC5(1.0);
```

Fig. 20. Jerarquías para el cuerpo del dragon.

Después definí cada modelo y escala, para el cuerpo le agregue a la translación la variable movD en X para que el dragón avance, y en la rotación le asigne la variable giro, para que el modelo gire en la dirección a la que se dirige. Utilice la jerarquía modelaux para colocar las alas, en estas ya no fue necesario colocar el movD en la translación, pero en el rotate utilice la función sen para que estas tuvieran el movimiento de aleteo, cada una con un signo diferente y un valor extra para que al momento de ejecutarse las alas no parecieran que chocaban cuando se alzaban.

```

model = glm::mat4(1.0); //Cuerpo de Tiamat
model = glm::translate(model, glm::vec3(movD+0.0f, 18.0f + (3 * sin(glm::radians(angulovaria * 5))), 30.0));
model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
modelaux = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(1.0f, 0.33f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cuerpo.RenderModel();

model = modelaux; //Ala 01 de Tiamat
model = glm::translate(model, glm::vec3(0.65f, 0.65f, 0.22f));
model = glm::rotate(model, 0.55f - sin(glm::radians(angulovaria * 3.5f)), glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(1.0f, 0.55f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Ala01.RenderModel();

model = modelaux; //Ala 02 de Tiamat
model = glm::translate(model, glm::vec3(0.65f, 0.65f, -0.22f));
model = glm::rotate(model, -0.55f + sin(glm::radians(angulovaria * 3.5f)), glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(1.0f, 0.55f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Ala02.RenderModel();

```

Fig. 21. Cuerpo y alas del dragon.

Después para cada cabeza ajuste su posición al cuerpo del dragon, a través de la jerarquía original, y al final guardando la información de cada una en una jerarquía diferente. Para que las cabezas realizaran el movimiento de cada función le agregue 2 rotate, a cada uno le asigne el valor de X y Y que se obtienen en las funciones anteriores. También le asigne su color de acuerdo con el orden que se dio en las instrucciones de la actividad.

```

model = modelaux; //Cabeza 01 de Tiamat
model = glm::translate(model, glm::vec3(-1.18f, 0.325f, 0.025f));
model = glm::rotate(model, glm::radians(espiralX), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(espiralY), glm::vec3(0.0f, 0.0f, 1.0f));
JerC1 = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cabeza01.RenderModel();

model = modelaux; //Cabeza 02 de Tiamat
model = glm::translate(model, glm::vec3(-0.765f, 0.075f, 0.6f));
model = glm::rotate(model, glm::radians(rosaX), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(rosaY), glm::vec3(0.0f, 0.0f, 1.0f));
JerC2 = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cabeza02.RenderModel();

```

Fig. 22. Movimiento de cabezas.

```

model = modelaux; //Cabeza 03 de Tiamat
model = glm::translate(model, glm::vec3(-0.765f, 0.0f, -0.68f));
model = glm::rotate(model, glm::radians(lemniscataX), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(lemniscataY), glm::vec3(0.0f, 1.0f, 0.0f));
Jerc3 = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cabeza03.RenderModel();

model = modelaux; //Cabeza 04 de Tiamat
model = glm::translate(model, glm::vec3(-0.6f, 1.15f, -0.55f));
model = glm::rotate(model, glm::radians(cardioideX), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(cardioideY), glm::vec3(0.0f, 1.0f, 0.0f));
Jerc4 = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cabeza04.RenderModel();

model = modelaux; //Cabeza 05 de Tiamat
model = glm::translate(model, glm::vec3(-0.7f, 1.12f, 0.52f));
model = glm::rotate(model, glm::radians(hipocicloideX), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(hipocicloideY), glm::vec3(0.0f, 1.0f, 0.0f));
Jerc5 = model;
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
color = glm::vec3(0.545f, 0.271f, 0.075f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Tiamat_Cabeza05.RenderModel();

```

Fig. 23. Movimiento de cabezas.

- 3) Extra 5 puntos: Hacer que cada cabeza escupa/emita su aliento (por medio de texturas con traslucidez) al presionar una tecla: Roja = Fuego, Azul = Rayos, Verde = Gas Venenoso, Blanco = Hielo, Café = Arena.

Para esta actividad algunas de las texturas para cada aliento los descargues de internet, pero la textura de hielo la realicé yo con unos dibujos de hielo que encontré. En el código main primero declare las variables y las texturas para esta actividad, además agregue una textura que está en blanco la cual es se mostrar primero cuando las cabezas no muestran aliento.

```

//Actividad Extra - Efecto de Alientos
float toffsetAlientou = 0.0f;
float toffsetAlientov = 0.0f;
float toffsetAlientocambiau = 0.0;

Texture gasTexture;
Texture fuegoTexture;
Texture rayoTexture;
Texture gasVTexture;
Texture hieloTexture;
Texture arenaTexture;
Texture NothingTexture;

```

Fig. 24. Variables para la actividad.

Después definí una nueva figura en el código, un rectángulo para colocar las texturas, así como en el letrero. También definí la ubicación de las texturas en la carpeta del proyecto.

```

unsigned int elementosIndices[] = { 0,1,2, 0,2,3, };
GLfloat elementosVertices[] = {
    -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f,
    0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, 1.0f,
    0.5f, 0.5f, 0.0f, 1.0f, 1.0f, 1.0f, 0.0f, 1.0f,
    -0.5f, 0.5f, 0.0f, 0.0f, 1.0f, 1.0f, 1.0f, 1.0f, };
Mesh* obj8 = new Mesh();
obj8->CreateMesh(elementosVertices, elementosIndices, 32, 6);
meshList.push_back(obj8);

```

Fig. 25. Rectángulo para los alientos.

```

gasTexture = Texture("Texturas/gas.png");
gasTexture.LoadTextureA();
fuegoTexture = Texture("Texturas/Fuego.png");
fuegoTexture.LoadTextureA();
rayoTexture = Texture("Texturas/Rayo.png");
rayoTexture.LoadTextureA();
gasVTexture = Texture("Texturas/GasV.png");
gasVTexture.LoadTextureA();
hieloTexture = Texture("Texturas/Hielo.png");
hieloTexture.LoadTextureA();
arenaTexture = Texture("Texturas/Arena.png");
arenaTexture.LoadTextureA();
NothingTexture = Texture("Texturas/Nothing.tga");
NothingTexture.LoadTextureA();

```

Fig. 26. Ubicación de modelos y textura.

En los archivos Window.cpp y Window.h definí las funciones para que con las teclas de la 1 al 5 se ejecutaran los alientos correspondientes,

```

bool getF() { return fuego; };
bool getT() { return trueno; };
bool getG() { return gas; };
bool getH() { return hielo; };
bool getA() { return arena; };

```

Fig. 27. Funciones públicas para los alientos en Window.h.

```

bool fuego = false;
bool trueno = false;
bool gas = false;
bool hielo = false;
bool arena = false;

```

Fig. 28. Funciones privadas para los alientos en Window.h.

```

if (key == GLFW_KEY_1 && action == GLFW_PRESS){theWindow->fuego = !theWindow->fuego;}
if (key == GLFW_KEY_2 && action == GLFW_PRESS){theWindow->trueno = !theWindow->trueno;}
if (key == GLFW_KEY_3 && action == GLFW_PRESS){theWindow->gas = !theWindow->gas;}
if (key == GLFW_KEY_4 && action == GLFW_PRESS){theWindow->hielo = !theWindow->hielo;}
if (key == GLFW_KEY_5 && action == GLFW_PRESS){theWindow->arena = !theWindow->arena;}

```

Fig. 29. Declaración de teclas para los alientos en Window.cpp.

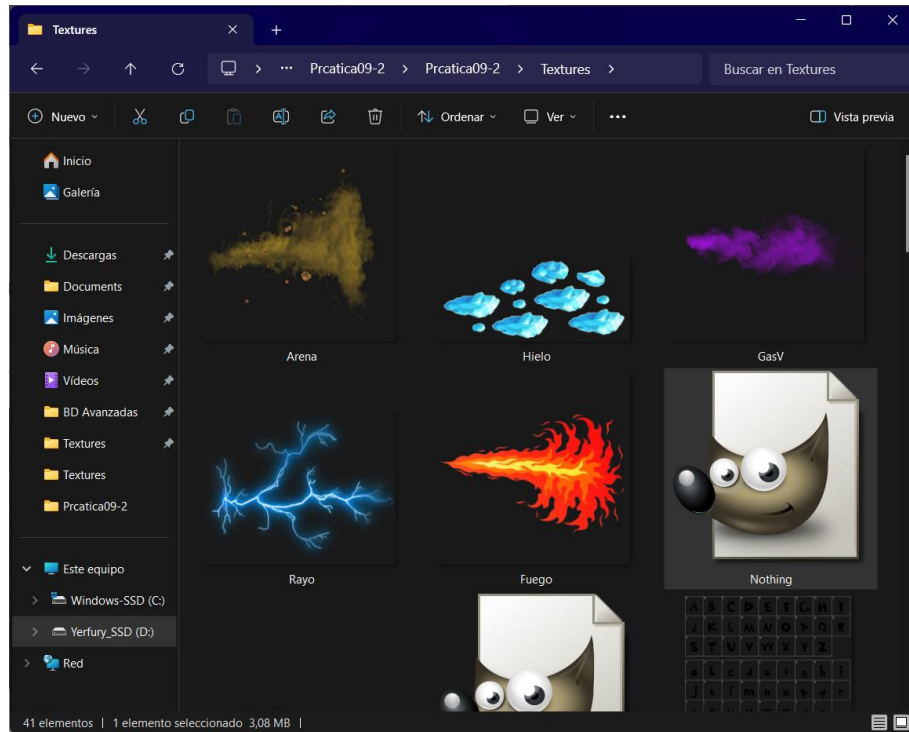


Fig. 30. Texturas de los alientos.

Después en el archivo main definí la ubicación y posición de cada aliento de acuerdo con la cabeza, a través de la jerarquía que define para cada una al inicio, principalmente utilizando rotates en cada eje para acomodarlo como si salieran de la boca de cada cabeza, además como efecto extra agregué el movimiento a la textura para que pareciera que el aliento salda de la boca de cada cabeza.

Para controlar el efecto de que sale o no el aliento utilizo la textura nothing, a través de if else, si se presiona la tecla del aliento respectivo se mostrara la textura de del efecto, un valor true, pero si no se teclea nada el valor es false y la textura es transparente. La tecla 1 controla la textura de fuego, la tecla 2 controla la textura de rayo, la tecla 3 controla la textura de gas venenoso, la tecla 4 controla la textura de hielo y la tecla 5 controla la textura de arena.

```
toffsetAlientocambiau -= 0.01 * deltaTime;
if (toffsetAlientocambiau > 1.0) toffsetAlientocambiau = 0.0;
toffsetAlientov = 0.0;
toffset = glm::vec2(toffsetAlientocambiau, toffsetAlientov);
```

Fig. 31. Velocidad de animación de aliento.


```

model = JerC1;//Aliento de FUEGO
model = glm::translate(model, glm::vec3(-3.5f, -0.3f, -1.15f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -20 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 215 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 0.5f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (mainWindow.getF()) { fuegoTexture.UseTexture(); }
else { NothingTexture.UseTexture(); }
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();

model = JerC2;//Aliento de RAYO
model = glm::translate(model, glm::vec3(-3.4f, -2.2f, 1.9f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 34 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 215 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 0.5f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (mainWindow.getT()) { rayoTexture.UseTexture(); }
else { NothingTexture.UseTexture(); }
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();

```

Fig. 32. Ajuste de alientos.

```

model = JerC3;//Aliento de GAS VENENOSO
model = glm::translate(model, glm::vec3(-3.4f, 0.15f, -1.9f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -25 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 195 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 0.8f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (mainWindow.getG()) { gasVTexture.UseTexture(); }
else { NothingTexture.UseTexture(); }
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();

model = JerC4;//Aliento de Hielo
model = glm::translate(model, glm::vec3(-2.95f, 1.32f, -2.8f));
model = glm::rotate(model, 10 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 0.4f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (mainWindow.getH()) { hieloTexture.UseTexture(); }
else { NothingTexture.UseTexture(); }
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();

model = JerC5;//Aliento de Arena
model = glm::translate(model, glm::vec3(-1.85f, 1.7f, 3.0f));
model = glm::rotate(model, -100 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 30 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 15 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 0.5f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (mainWindow.getA()) { arenaTexture.UseTexture(); }
else { NothingTexture.UseTexture(); }
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[7]->RenderMesh();

```

Fig. 33. Ajuste de alientos.

Con todas esas modificaciones, la ejecución del ejercicio fue la siguiente:



Fig. 24. Ejecución del ejercicio.



Fig. 25. Ejecución del ejercicio.

Se adjunto a la entrega el video donde se muestra la ejecución del código.

2. Problemas presentados.

Para estas actividades no tuve ningún problema, la aplicación de estas actividades me fue fácil de entender debido a que entendí como realizar el de la practica anterior, considero que fueron mas laboriosas, si necesite bastante tiempo para realizarlas, asegurándome mas que nada que la posición de los elementos fuera correcta.

3. Conclusión:

Tras realizar los ejercicios de la practica me parece que la complejidad era normal, pero el requerido para realizar cada una era bastante, eran actividades laboriosas, la implementación movimientos por ecuaciones me pareció una forma interesante de realizar movimientos que podrían parecer aleatorios, pero en realidad si tiene una secuencia o comportamiento, además la implementación de los movimientos en las texturas me pareció un efecto muy interesante de implementar. Al final considero que esta actividad fue un buen complemento para implementar animaciones más complejas en el ambiente o dar la ilusión de las texturas se mueven, siendo esto a mi parecer una buena práctica para el desarrollo del proyecto final.

Bibliografía.

- INTRODUCCIÓN A LAS ESPIRALES. Ieshumanes. <http://www.ieshumanes.com/nw/Espiral/espirales.pdf>. Accedido octubre 24, 2024.
- Wolfram Research, Inc. Rose Curve. Wolfram MathWorld. <https://mathworld.wolfram.com/RoseCurve.html>. Accedido octubre 24, 2024.
- Lemniscate. Virtualmathmuseum. <https://virtualmathmuseum.org/docs/Lemniscate.pdf>. Accedido octubre 24, 2024.
- Bourne M. Lemniscate of Bernoulli. Interactive Mathematics. <https://www.intmath.com/historical-math-curves/lemniscate-bernoulli.php>. Accedido octubre 24, 2024.
- Equation of Cardioid/Parametric. Proofwiki. https://proofwiki.org/wiki/Equation_of_Cardioid/Parametric#google_vignette. Accedido octubre 24, 2024.
- Cycloid - Definition, Equations, Area, and Curve. Math Monks. <https://mathmonks.com/cycloid>. Accedido octubre 24, 2024.